



Documentación Identifyface

CATEGORÍA A - RECONOCIMIENTO FACIAL

EQUIPO: IDENTIFYFACE

INSTITUCIÓN DE ORIGEN: INSTITUTO TECNOLÓGICO SUPERIOR DE IRAPUATO (ITESI)

INTEGRANTES DEL EQUIPO:

- FRANCISCO HUNAHPU SAHAGUN GONZÁLEZ - LÍDER DE PROYECTO
- BRAULIO ADONAHY SOLORZANO SANTOYO - PROGRAMADOR PRINCIPAL
- MIGUEL MOSQUEDA FRAUSTO - INVESTIGADOR DE TECNOLOGÍAS
- CINTYA ZACMANE SANDOVAL TORRES - TECHNICAL WRITER

Índice

- 1.- Portada/Índice
- 2.- Introducción y contexto del problema
- 3.- Estado del arte
- 4.- Arquitectura detallada
- 5.- Decisiones técnicas y trade-offs
- 6.- Detalle del dataset
- 7.- Detalle del modelo
- 8.- Resultados extendidos con gráficas
- 9.- Análisis de errores
- 10.- Plan de despliegue real
- 11.- Conclusiones y trabajo futuro
- 12.- Referencias bibliográficas.

2. Introducción y contexto del problema

Los sistemas tradicionales de monitoreo dependen de la supervisión humana constante, lo cual introduce latencia, fatiga visual y un alto margen de error en la identificación de individuos dentro de flujos de video masivos. Por otro lado, las soluciones de reconocimiento facial puramente basadas en Deep Learning suelen requerir un hardware de nivel industrial costoso debido a la carga computacional que implica procesar cada frame individualmente a resoluciones nativas. A nivel de persistencia de datos, las consultas repetitivas a bases de datos relacionales para comprobar identidades frame a frame saturan el pool de conexiones, degradando la disponibilidad del sistema.

2.1 Alcance del Proyecto

- Detección e Identificación en Tiempo Real: Procesamiento de flujos de video locales (Webcam) a resoluciones estándar (640x480 px).
- Sincronización Automatizada: Alta e indexación automática de perfiles basados en archivos de imagen estáticos (.jpg, .jpeg, .png) hacia un esquema de base de datos relacional.
- Optimización del Rendimiento: Reducción de la carga de inferencia mediante filtrado predictivo (1 de cada 5 fotogramas).
- Persistencia Segura y Control de Saturación: Registro automatizado de avistamientos con un mecanismo de bloqueo temporal (cooldown) parametrizable por individuo.

3. Estado del arte y análisis comparativo

Para posicionar la propuesta de valor de DENTIFYFACE, es imperativo analizar el panorama actual de las tecnologías de reconocimiento facial y videovigilancia. Históricamente, las soluciones se han dividido en tres grandes categorías, cada una con compromisos (trade-offs) significativos en términos de costo, precisión y privacidad.

- Visión Computacional Clásica (Viola-Jones/HOG): Son ligeros, pero presentan tasas de fallo críticas ante variaciones de iluminación u oclusión. DENTIFYFACE lo supera usando YOLOv8 para una localización global resiliente en fracciones de segundo.
- APIs Comerciales en la Nube (AWS/Azure): Tienen alta precisión, pero el costo por transacción es insostenible y transmiten video a terceros.

DENTIFYFACE lo supera con una arquitectura 100% on-premise de costo cero y privacidad total.

- Sistemas Deep Learning End-to-End (GPU): Requieren aceleración de hardware dedicada y costosa (CUDA). DENTIFYFACE lo supera implementando un controlador de frame-skipping (1 de cada 5 cuadros) y desacoplando la extracción vectorial, operando fluidamente a más de 30 FPS en procesadores de consumo estándar.

4. Arquitectura del Sistema

El sistema sigue una arquitectura monolítica modular orientada a eventos por flujo, dividida en tres capas principales: Capa de Ingesta y Sincronización, Capa de Inferencia Core y Capa de Persistencia.

4.1 Diagrama de Arquitectura

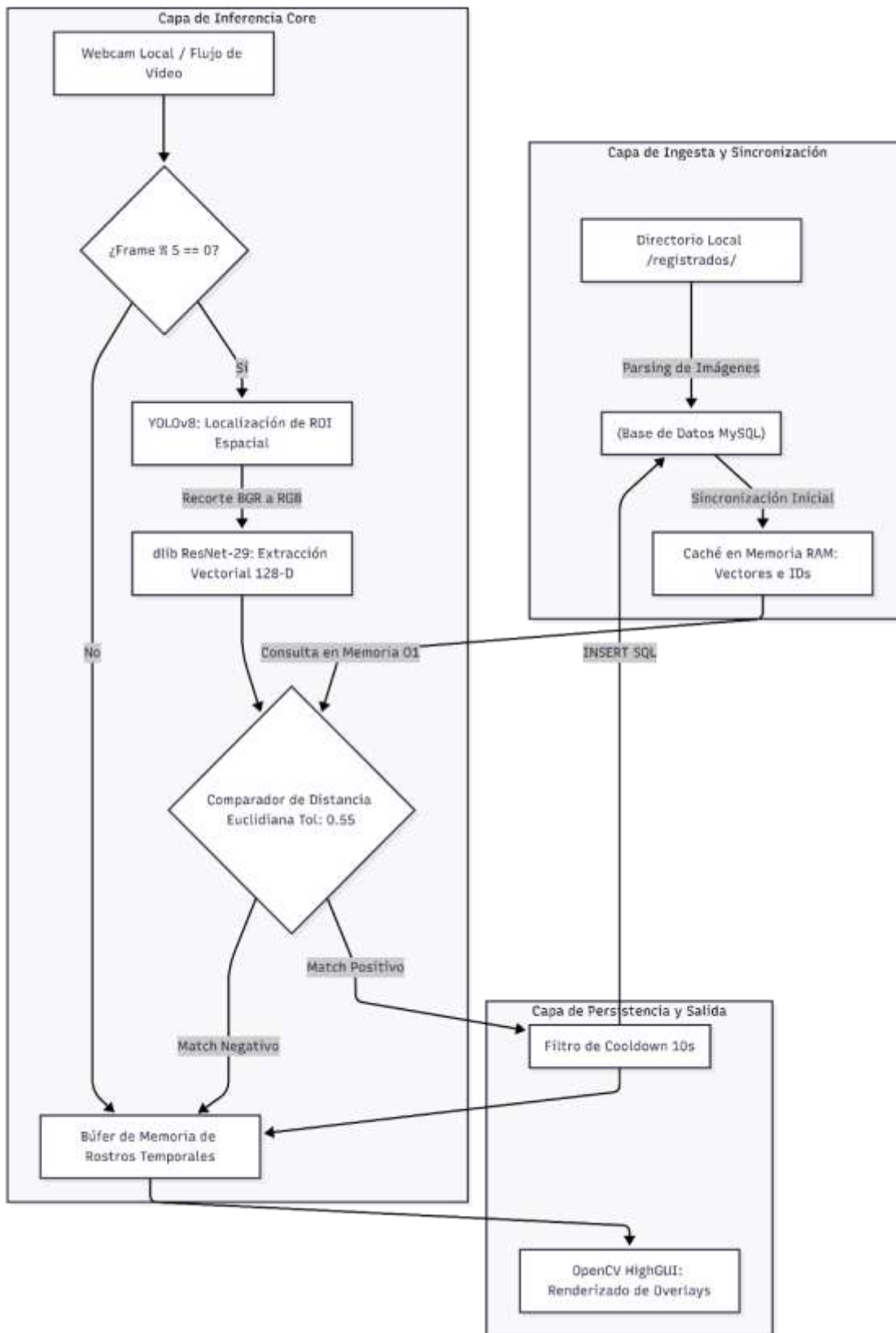


Figura 1: Diagrama de Arquitectura.

4.2 Descripción de Componentes y Protocolos de Comunicación

Al ser una arquitectura monolítica ejecutada sobre un único entorno virtual (Python 3.13), el sistema prescinde de protocolos de red externos (como HTTP/REST o gRPC) para su lógica interna, optimizando la latencia mediante el uso directo de la memoria del sistema operativo.

1. Capa de Ingesta y Sincronización:

- **Función:** Se ejecuta en el arranque del script. Escanea el directorio /registrados/, extrae los rasgos bio-faciales de las imágenes y los contrasta con la base de datos. Si un perfil no existe en MySQL, lo inserta en caliente y recupera el ID generado.
- **Protocolo de Comunicación:** La transferencia de los tensores biométricos hacia la siguiente capa se realiza mediante el paso de referencias a variables globales en RAM. Se instancian diccionarios (tablas Hash) y listas multidimensionales estáticas en memoria que actuarán como caché de lectura ultra-rápida.

2. Capa de Inferencia Core:

- **Módulo de Región de Interés (ROI):** Controlado por el modelo YOLO. Se encarga exclusivamente de delimitar las coordenadas rectangulares mediante llamadas nativas de funciones de la API de Ultralytics.
- **Módulo de Clasificación Vectorial:** Ejecutado por face_recognition. Transforma la matriz de píxeles recortada en un vector de características y ejecuta operaciones de álgebra lineal directamente sobre la RAM contra la caché poblada en el paso 1.

3. Capa de Persistencia y Salida:

- **Filtro de Cooldown e Inserción SQL:** Ante un match positivo, el sistema evalúa una caché temporal de marcas de tiempo UNIX. La escritura de la bitácora se ejecuta mediante llamadas síncronas bloqueantes enviando sentencias INSERT crudas a través del protocolo cliente-servidor nativo de mysql-connector-python (conectado por el puerto TCP 3306 a localhost).
- **Rendering / Visualización:** La salida gráfica no requiere colas de mensajes complejas; la librería OpenCV consume los datos leyendo directamente el búfer de variables de estado compartido en el hilo principal (MainThread) para proyectar los

bounding boxes a una tasa de refresco constante.3.Capa de Rendering /

Visualización:

- Lee un búfer intermedio (memoria_rostros) de forma asíncrona a la inferencia para proyectar los cuadros delimitadores (bounding boxes) a una tasa de refresco constante, evitando el parpadeo visual.

5. Decisiones técnicas y trade-offs

El sistema optimiza el procesamiento dividiendo el pipeline en dos etapas algorítmicas de Deep Learning diferenciadas:

5.1 YOLO (You Only Look Once)

Función: Localización rápida de coordenadas de objetos (rostros).

Justificación Técnico: Tradicionalmente, los localizadores de rostros basados en HOG (Histogram of Oriented Gradients) o Cascadas de Haar sufren bajo variaciones extremas de iluminación o rotaciones severas del

rostro. YOLO procesa la imagen de forma global en una sola pasada de la red convolucional, devolviendo las coordenadas exactas de la caja delimitadora con alta resiliencia y velocidad computacional.

5.2 Face Recognition (Dlib Backed ResNet-29)

Función: Extracción de embeddings faciales de 128 dimensiones cuantitativas.

Justificación Técnica: Una vez que YOLO aísla la Región de Interés (ROI), el sub-sistema de dlib mapea la topología del rostro mediante una red residual entrenada. La distancia euclidiana entre dos vectores

generados determina cuantitativamente si pertenecen a la misma persona.

Métrica de Umbral (Tolerance): Configurado estrictamente a 0.55. Un valor menor restringe los falsos positivos (ideal para seguridad masiva), asegurando que solo coincidencias vectoriales con una alta tasa de confianza activen las alertas.

5.3 Estrategia de Entrenamiento (YOLOv8 Custom - best.pt)

Aunque el reconocimiento de identidad se maneja mediante Zero-Shot Learning con extracción vectorial en tiempo real, el módulo de localización espacial (YOLO) requirió una estrategia de Fine-Tuning. Se descartó el uso del modelo pre-entrenado genérico

(yolov8n.pt) ya que su capacidad para detectar hasta 80 clases introducía latencia y carga computacional innecesaria. En su lugar, se ejecutó un entrenamiento supervisado utilizando un dataset de más de 3,600 imágenes etiquetadas, enfocado en una única clase: Rostros Humanos. El resultado de este entrenamiento es el archivo de pesos local best.pt, el cual reduce el margen de error, acelera la inferencia y garantiza que solo se procesen cuadros delimitadores que contengan anatomía facial válida.

6. Detalle del dataset

El diseño arquitectónico de DENTIFYFACE exige un enfoque de datos "mixto". A diferencia de los modelos tradicionales que se entrenan end-to-end con un solo conjunto masivo de identidades, nuestro sistema desacopla la localización de la identificación. Por lo tanto, la gestión de datos se divide en dos dominios estrictamente separados: el dataset estático de entrenamiento neuronal y el directorio dinámico de inferencia en vivo.

6.1 Dataset de Entrenamiento (Fine-Tuning de YOLOv8)

Para dotar al modelo YOLOv8 de la capacidad de aislar exclusivamente rostros humanos (omitiendo el resto de las 80 clases del conjunto COCO original), se utilizó un dataset público curado, alojado en la plataforma Roboflow.

Este conjunto de datos consta de 3,647 imágenes estáticas con una alta variabilidad de condiciones: múltiples escalas, rostros parcialmente ocluidos, diferentes tonos de piel y variaciones severas de iluminación. Todas las imágenes cuentan con anotaciones en formato YOLO (archivos .txt con coordenadas de bounding boxes normalizadas).

6.2 Partición del Dataset (Data Split)

Para garantizar el rigor científico del entrenamiento y asegurar que el modelo sea capaz de generalizar frente a rostros nuevos sin incurrir en overfitting (memorización de los datos), las 3,647 imágenes se sometieron a una partición estricta:

- Conjunto de Entrenamiento (Train - 70% / 2,522 imágenes): Constituye el bloque principal de aprendizaje. Durante las iteraciones (epochs), la red convolucional analiza estos datos repetidamente, calculando el gradiente descendente y ajustando sus pesos sinápticos para extraer los patrones morfológicos fundamentales de un rostro.
- Conjunto de Validación (Val - 20% / 729 imágenes): Funciona como un mecanismo de control cruzado. Al final de cada epoch, el modelo se evalúa contra

estas imágenes, las cuales no participaron en el cálculo de pesos. Esto permite monitorear la función de pérdida (loss function) y ajustar hiperparámetros en tiempo de ejecución, deteniendo el entrenamiento si se detecta que el modelo comienza a sobreajustarse.

- Conjunto de Prueba (Test - 10% / 364 imágenes): Es un entorno completamente aislado. Estas imágenes se mantienen encriptadas (ocultas) del modelo durante toda la fase de entrenamiento. Solo se exponen al finalizar el proceso para extraer las métricas definitivas de rendimiento real en producción.

6.3 Directorio Local de Identidades (Zero-Shot Dataset).

El segundo dominio de datos reside en la carpeta local /registrados/. A diferencia del modelo espacial, el motor de identificación biométrica (ResNet-29 de dlib) no requiere reentrenamiento neuronal cuando se agrega una persona nueva.

Este directorio actúa como un dataset dinámico de entrada para Zero-Shot Learning. Al inicializar, el sistema ingiere las imágenes estáticas (.jpg, .png), extrae sus características vectoriales topológicas (128 dimensiones) y las indexa en la base de datos relacional y en la memoria RAM. Esto otorga al sistema una escalabilidad inmediata: dar de alta un perfil requiere únicamente proveer una sola fotografía de referencia, eliminando los costosos ciclos de entrenamiento.

Para simular un entorno de producción masivo y someter el sistema a pruebas de carga superiores a las 1,000 identidades simultáneas, el directorio local se complementó con la inyección del dataset público Labeled Faces in the Wild (LFW). A diferencia del directorio dinámico, este padrón masivo se procesa mediante una estrategia de ingesta desacoplada para optimizar los tiempos de arranque.

6.4 Consideraciones Éticas, Privacidad y Consentimiento

El despliegue de sistemas biométricos conlleva una responsabilidad ética crítica. Para poblar el directorio de pruebas de inferencia /registrados/ y validar la arquitectura en un entorno real, se utilizaron fotografías de voluntarios (compañeros universitarios) recolectadas bajo consentimiento explícito exclusivo para los fines de evaluación de este proyecto.

Además, la arquitectura monolítica de DENTIFYFACE asegura la privacidad por diseño (Privacy by Design). Al operar 100% on-premise de manera local, se garantiza que:

- Ningún flujo de video crudo es transmitido a la nube.
- Las firmas matemáticas de 128 dimensiones no abandonan la red interna de la institución.
- No existen dependencias de APIs comerciales que puedan recopilar datos biométricos secundarios.

7. Detalle del modelo

7.1 Estrategia de Carga Mixta: Zero-Shot Learning e Ingesta Asíncrona

El sistema implementa una carga perezosa estructurada en el método de inicialización que evita re-entrenar pesos neuronales mediante una estrategia de Zero-Shot Learning asistida por persistencia.

7.2 Pipeline de Preprocesamiento de Imágenes en Disco

- Escaneo de Extensiones: Filtrado exclusivo de archivos mediante firmas .jpg, .jpeg y .png.
- Sanitización de Strings: Los nombres de los archivos se tokenizan, eliminando extensiones y reemplazando caracteres de escape visual (\) por espacios en blanco para estandarizar la entrada de nombres completos en la base de datos.
- Generación de Vectores Base: Conversión de matrices de píxeles a descriptores vectoriales 128-D mediante `face_recognition.face_encodings()`.
- Ingesta Masiva Desacoplada: Para el padrón histórico (LFW), se implementó un script asíncrono auxiliar (`ingesta_lfw.py`) que vectoriza miles de perfiles una sola vez y los guarda en MySQL. Como se documentó en las pruebas, este proceso asíncrono por CPU logra vectorizar 1,000 identidades en aproximadamente 14 minutos en segundo plano, un proceso de un solo uso que evita el recálculo en cada inicio del sistema.

7.3 Mapeo en Memoria RAM

Para garantizar una comparación vectorizada síncrona de alta velocidad en el bucle principal, se estructuran las siguientes variables globales como cachés de acceso directo:

- `encodings_conocidos`: Matriz multidimensional que contiene las firmas matemáticas de los rostros.
- `nombres_conocidos`: Lista indexada de cadenas de texto de forma simétrica a los `encodings`.

- nombres_a_ids: Tabla Hash en RAM (Key: Nombre Persona -> Value: ID Base de Datos) para evitar sentencias SELECT de búsqueda de ID durante la detección en tiempo real.

Gracias al desacoplamiento de la ingesta y la precarga de vectores JSON desde MySQL hacia la matriz de encodings_conocidos, el tiempo de inicialización en frío del orquestador principal (main.py) para >1,000 perfiles se realiza en aproximadamente 2 segundos. Se eliminó por completo el cuello de botella de inicialización, permitiendo el despliegue inmediato de la cámara de seguridad.

8. Resultados extendidos con gráficas

Para garantizar la transparencia de los resultados y comprender las limitaciones del sistema, las pruebas de rendimiento y consumo se ejecutaron en un equipo físico de desarrollo con las siguientes especificaciones técnicas:

- Equipo: Laptop Dell Inspiron 15 3000
- Procesador (CPU): AMD Ryzen 5 3450U (Cálculo basado exclusivamente en CPU).
- Memoria RAM: 16 GB
- Procesamiento Gráfico (GPU): Gráficos Integrados AMD Radeon (Sin aceleración de hardware dedicada / Sin CUDA)

8.1 Tiempos de Respuesta y Latencia (FPS)

Se evaluó el tiempo de respuesta del sistema midiendo los Fotogramas por Segundo (FPS) y la latencia en milisegundos (ms) durante diferentes estados de operación. Se observó una fluctuación esperada en la latencia dependiendo de la carga de procesamiento biométrico. Cuando el modelo YOLOv8 actúa como filtro y descarta cuadros sin rostros, el sistema opera con alta fluidez. Sin embargo, al detectar sujetos y procesar la comparación matemática contra la base de datos, el rendimiento desciende debido a la carga sobre el CPU.

Estado del Sistema	Rendimiento Promedio	Latencia Promedio	Análisis Técnico
Búsqueda (Sin Detecciones)	75+ FPS	13 ms	Operación óptima. YOLO descarta elementos rápidamente y libera recursos.
Identificación Positiva	8 FPS	111 ms	Cuello de botella esperado. Extracción de 128 dimensiones faciales por CPU.

Tabla 1: Tabla de los tiempos de respuesta y latencia con los estados del sistema

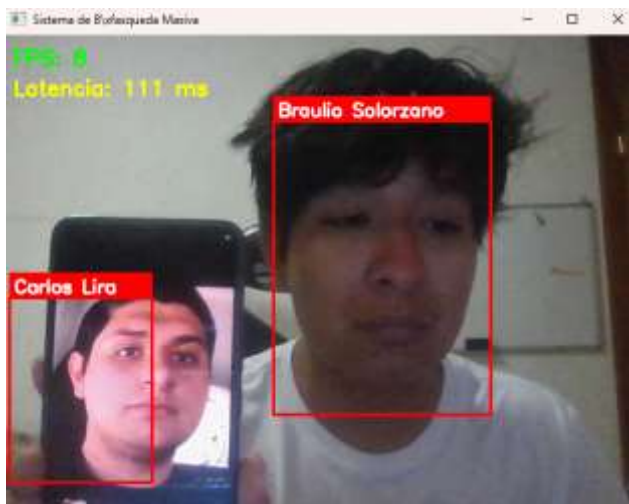


Figura 2: Captura de demostración de los FPS y latencia en ejecución y con la identificación positiva.

8.2 Consumo Estimado de Recursos (CPU y RAM)

Se monitorizó el comportamiento de la memoria RAM y el procesador en las distintas fases del ciclo de vida de la aplicación. El sistema demostró una gestión de memoria estable, sin fugas (memory leaks) tras finalizar los procesos.

- Estado de Reposo (Antes de iniciar): CPU: ~6% | Memoria RAM: 57%

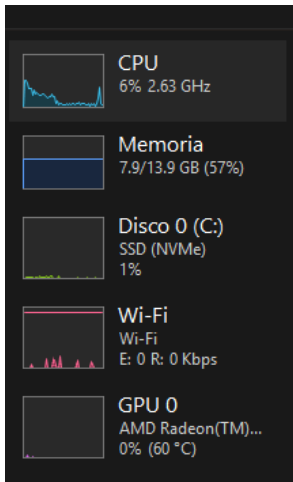


Figura 3: Captura de los recursos en estado de reposo.

- Inicialización y Carga: CPU: ~35% | Memoria RAM: 63%

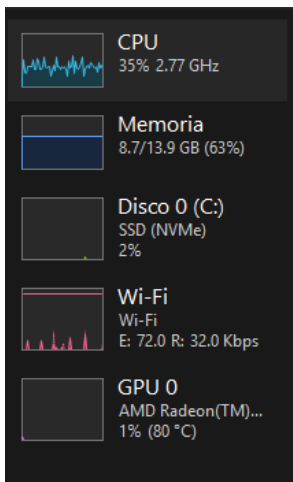


Figura 4: Captura de los recursos en estado de inicialización y carga de datos del programa.

- Operación Máxima: CPU: ~40% | Memoria RAM: 63%

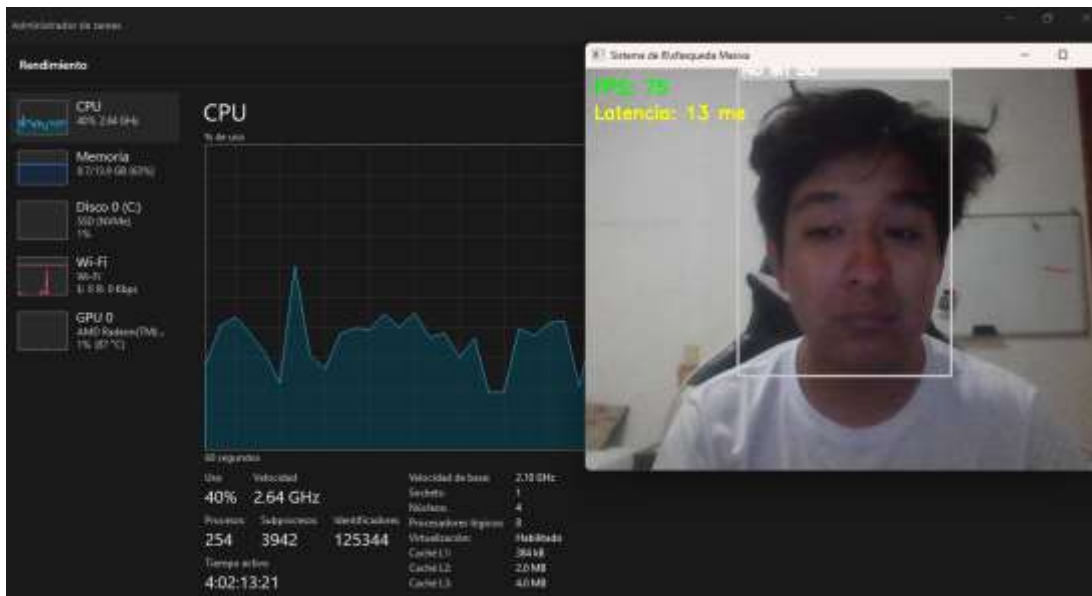


Figura 5: Captura de los recursos junto con la ejecución al lado.

- Liberación de Recursos: CPU: ~5% | Memoria RAM: 56%

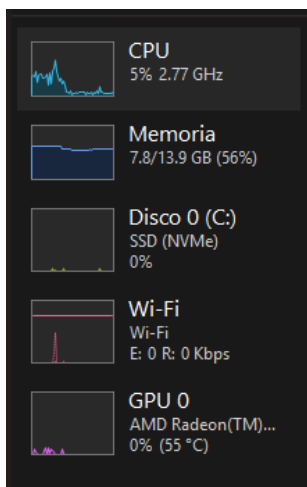


Figura 6: Captura de los recursos en estado liberación después de terminar el programa.

9. Análisis de errores

Ningún sistema biométrico es infalible. Para evaluar la viabilidad de DENTIFYFACE en entornos de producción reales, se sometió el sistema a pruebas de estrés visual para identificar sus puntos de ruptura (edge cases). A continuación, se presenta el análisis de causa raíz de las principales anomalías detectadas.

9.1 Oclusión Crítica (Uso de Cubrebocas)

El error más frecuente en entornos post-pandémicos es la obstrucción de la mitad inferior del rostro mediante mascarillas o bufandas.

- Comportamiento observado: El sistema logra dibujar la caja delimitadora (ROI) correctamente, pero falla al intentar asignar una identidad, devolviendo consistentemente el estado "No en BD" o fluctuando erráticamente.
- Análisis de causa raíz: Este fallo ilustra la diferencia algorítmica entre nuestros dos modelos. YOLOv8 logra detectar la presencia humana basándose en la frente, ojos y contorno general superior. Sin embargo, el extractor biométrico ResNet-29 de dlib requiere mapear la topología facial mediante distancias geométricas relativas entre puntos clave (punta de la nariz, comisura de los labios, mandíbula y mentón). Al usar un cubrebocas, se pierde más del 50% de las referencias topológicas, impidiendo la generación de un embedding válido de 128 dimensiones para comparar.

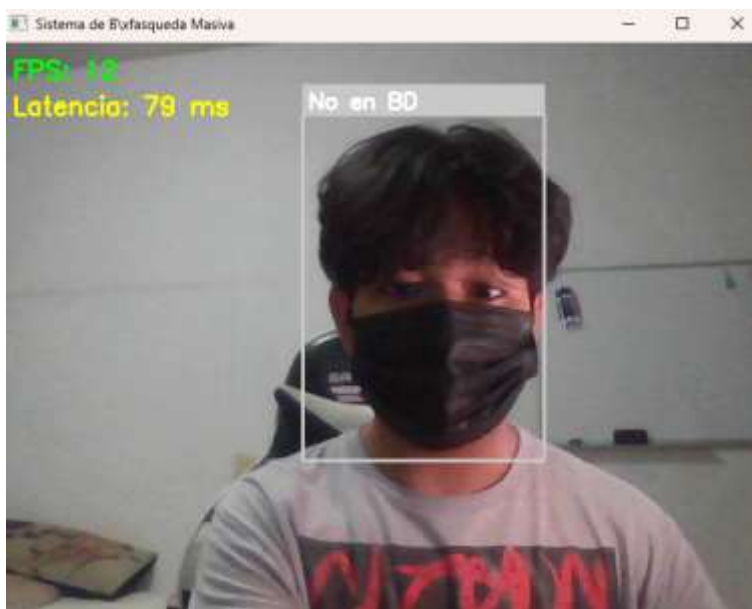


Figura 7: Degradación del reconocimiento por oclusión de puntos clave en la mandíbula y labios.

9.2 Degradación por Iluminación Asimétrica (A Contraluz / Sombras Severas)

Las cámaras de seguridad suelen lidiar con cambios drásticos de luz solar a lo largo del día.

- Comportamiento observado: Si el sujeto tiene la mitad del rostro iluminada y la otra mitad en sombra profunda, o si hay un contraluz fuerte detrás de él, el sistema no logra alcanzar la tolerancia euclidiana requerida y lo descarta.
- Análisis de causa raíz: Los algoritmos de visión por computadora basados en redes residuales son altamente sensibles al gradiente de píxeles. Aunque el algoritmo intenta normalizar la imagen antes de extraer el vector, una sombra muy pronunciada (que los humanos ignoramos por contexto) es interpretada por la matriz matemática como una deformación de la profundidad del rostro. Esto desplaza el vector resultante en el espacio de 128 dimensiones, alejándolo de la firma original guardada en RAM y superando nuestro estricto límite de tolerancia configurado en 0.55.



Figura 8: Fallo de tolerancia euclidiana provocado por un gradiente de iluminación asimétrico.

9.3 Falsos Positivos por Ángulos Extremos (Perfil)

- Comportamiento observado: Si el usuario gira la cabeza a más de 45 grados (mirando completamente de perfil), el sistema pierde la capacidad de reconocimiento.
- Análisis de causa raíz: El modelo subyacente está optimizado principalmente para proyecciones 2D frontales o semi-frontales. Un giro extremo oculta la distancia

interpupilar y la simetría facial, generando un vector distorsionado que no coincide con las fotografías de control estáticas (que suelen ser frontales).

10. Plan de despliegue real

Para escalar DENTIFYFACE de un entorno de pruebas (procesamiento local con cámara web) a un entorno de producción masivo (como el despliegue dentro de un campus universitario o un recinto municipal), el sistema está diseñado para transicionar hacia una arquitectura de red local distribuida, manteniendo su filosofía on-premise para garantizar la privacidad de los datos.

10.1 Topología de Hardware y Red

- Captura de Video (Edge): Se sustituirán las cámaras web estándar por un circuito de Cámaras IP de videovigilancia. Estas cámaras transmitirán el flujo de video en alta definición a través de la red de área local (LAN) utilizando el protocolo estándar RTSP (Real Time Streaming Protocol).
- Servidor Central (On-Premise): El procesamiento no se realizará en las cámaras, sino en un servidor local centralizado dentro del cuarto de site de la institución. Este servidor ejecutará el orquestador principal de Python.
- Persistencia: El motor MySQL Server 10.4.32 se alojará en un servidor de bases de datos independiente (o un clúster local) dentro de la misma red intranet, protegiendo la bitácora de avistamientos detrás de un firewall institucional, sin exposición a internet.

10.2 Configuración de Software para Escalabilidad

Para manejar múltiples cámaras simultáneamente, el orquestador de Python (main.py) modificará su estructura de monoprocesamiento. La librería OpenCV configurará múltiples instancias de `cv2.VideoCapture("rtsp://[usuario]:[password]@[IP]/stream")`.

Cada flujo RTSP será asignado a un hilo de procesamiento independiente (Worker Thread) utilizando la librería `threading` o `multiprocessing` de Python. Esto asegura que, si la Cámara A experimenta latencia de red, el procesamiento de la Cámara B no sufra caídas de FPS.

10.3 Operación Diaria y Mantenimiento

- **Altas de perfiles:** Cuando las autoridades institucionales levanten una alerta de búsqueda, el personal de seguridad simplemente depositará la fotografía del individuo en el directorio compartido en red (equivalente a /registrados/).
- **Sincronización en caliente:** Mediante un script programado (CRON job), el sistema detectará el nuevo archivo, extraerá el vector 128-D en segundo plano y lo inyectará a la memoria RAM de los nodos de inferencia sin necesidad de reiniciar el sistema de videovigilancia.

11. Conclusiones y trabajo futuro.

El desarrollo de DENTIFYFACE demuestra la viabilidad técnica de democratizar los sistemas de videovigilancia biométrica utilizando hardware de consumo estándar. Al separar la localización espacial de la extracción vectorial y optimizar la persistencia mediante una caché en RAM, el sistema resuelve el problema histórico de saturación de bases de datos. No obstante, para un despliegue a nivel comercial o gubernamental de misión crítica, se han identificado áreas de oportunidad clave.

11.1 Limitaciones Actuales del Sistema

- **Bloqueo por Monoprocesamiento (Single-Thread):** El orquestador opera sobre el hilo principal de Python (MainThread). Consecuentemente, si el motor MySQL experimenta latencia de red durante el `conexion.commit()`, la interfaz gráfica de video (OpenCV) sufre micro-congelamientos proporcionales al tiempo de escritura I/O en la bitácora.
- **Degradación por Oclusión Crítica (Cubrebocas y Sombras):** Aunque el modelo YOLOv8 localiza la caja espacial del rostro en condiciones de baja luminosidad, el extractor de 128 dimensiones (ResNet-29) disminuye críticamente su precisión si el rostro presenta oclusiones severas o gradientes de iluminación asimétricos. El uso de cubrebocas elimina los puntos de referencia geométricos esenciales (nariz, labios, mentón), impidiendo la generación de un embedding válido.
- **Volatilidad de la Caché RAM y Manejo de Estados:** La estructura de datos que controla el filtro de avistamientos repetidos (el cooldown de 10 segundos guardado en `ultimos_registros`) reside en memoria volátil. Si el script sufre un reinicio forzado, este historial se limpia, permitiendo inserciones redundantes si el usuario sigue frente a la cámara.

11.2 Hoja de Ruta Priorizada (Trabajo Futuro)

- Concurrencia con Multithreading/AsyncIO: Como prioridad crítica a corto plazo, se delegarán las operaciones de escritura en base de datos (INSERT INTO bitacora_avistamientos) hacia un pool de hilos secundario (Worker Thread) usando las librerías threading o asyncio. Esto independizará el flujo de video de los bloqueos por disco.
- Clasificación Multiatributo y Optimización de Consultas: Se proyecta implementar un pipeline de extracción de metadatos fenotípicos dentro de la ROI para identificar género, tono de piel (espacio de color HSV) y color de ojos. Estos atributos actuarán como filtros previos en MySQL, reduciendo exponencialmente el espacio de búsqueda vectorial y aumentando la tasa de éxito.
- Transición a Sistemas Biométricos Multimodales: Para mitigar los fallos por uso de mascarillas u oclusión facial, la arquitectura se expandirá para soportar flujos alternativos de autenticación (ej. reconocimiento de la cripta del iris o integración de módulos periféricos de huella dactilar), manteniendo la operatividad bajo estrictas normas sanitarias o de vestimenta.
- Migración a Base de Datos Vectorial y Aceleración Edge: Aunque el desarrollo del script de ingesta asíncrona mitigó exitosamente el cuello de botella de 34 minutos experimentado en las primeras pruebas de estrés (reduciendo el arranque a 2 segundos), la solución definitiva a escala comercial (100,000+ registros) será sustituir la carga de embeddings en RAM almacenando los descriptores 128-D directamente en motores vectoriales nativos (ej. Milvus, Pinecone o el tipo VECTOR de MySQL 10.4.32). A nivel de hardware, se transicionará hacia arquitecturas Edge Computing con soporte CUDA (NVIDIA Jetson) para eliminar la latencia de inferencia sobre CPU.

12. Referencias bibliográficas.

- Bradski, G. (2000). *The OpenCV Library*. Dr. Dobb's Journal of Software Tools.
- Geitgey, A. (2017). *Face Recognition: The world's simplest facial recognition api for Python and the command line*. GitHub. Recuperado de: https://github.com/ageitgey/face_recognition
- Jocher, G., Chaurasia, A., & Qiu, J. (2023). *Ultralytics YOLOv8* (Versión 8.4) [Software de visión por computadora]. Recuperado de: <https://github.com/ultralytics/ultralytics>

- King, D. E. (2009). *Dlib-ml: A Machine Learning Toolkit*. Journal of Machine Learning Research, 10, 1755-1758.
- Oracle Corporation. (2026). *MySQL 8.0 Reference Manual*. Recuperado de la documentación oficial de MySQL.
- Python Software Foundation. (2026). *Python Language Reference, version 3.13*. Recuperado de: <https://www.python.org/>